CMPS453 Cloud Computing Resource Management

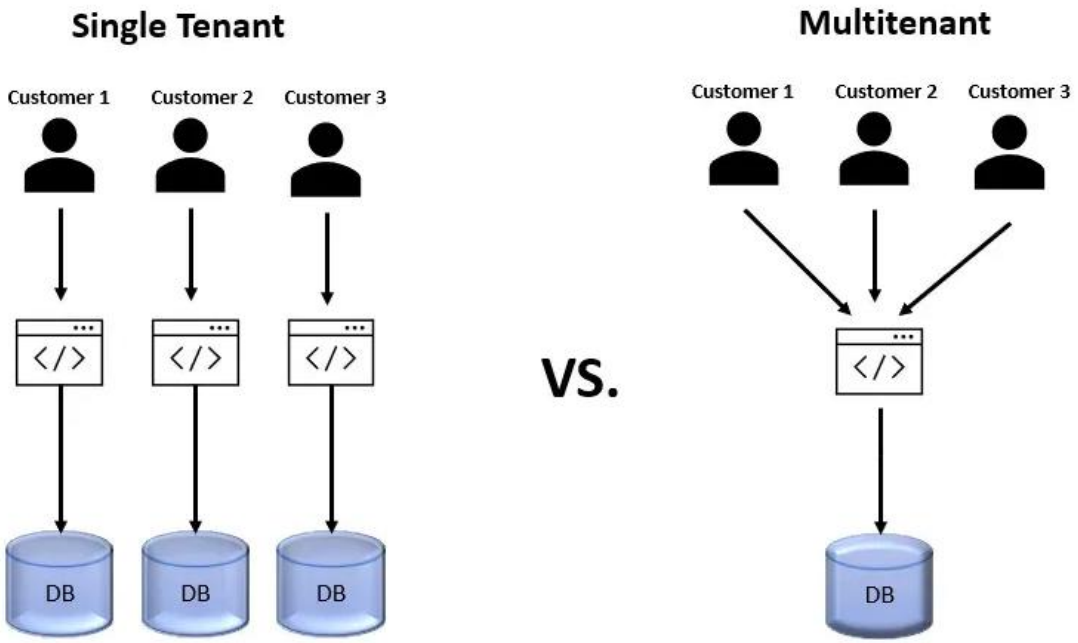
Ayman AboElHassan, PhD
Assistant Professor
ayman.abo.elmaaty@eng.cu.edu.eg

General Problem

1. VMs can not provide elastic computing by itself
   - If user created a VM with 8GB, how to increase the RAM when needed.

General Problem

1. VMs can not provide elastic computing by itself
   - If user created a VM with 8GB, how to increase the RAM when needed.
2. Hypervisor support sharing CPU/Memory, but not middleware platforms
   - If multiple users use MapReduce, how to manage which job deployed on which node.



General Problem

Users run a diverse of applications/frameworks

- Data streaming (MapReduce, Storm)
- ML training (Ray)
- Database (DynamoDB)
- Storage (HDFS)

Which servers to run HDFS?
And which for DynamoDB?

General Problem

Users run a diverse of applications/frameworks

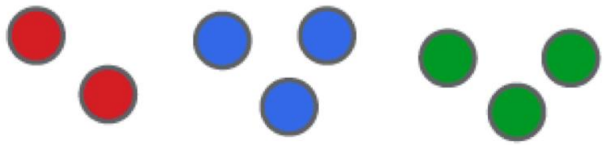
- Data streaming (MapReduce, Storm)
- ML training (Ray)
- Database (DynamoDB)
- Storage (HDFS)

Which servers to run HDFS?
And which for DynamoDB?

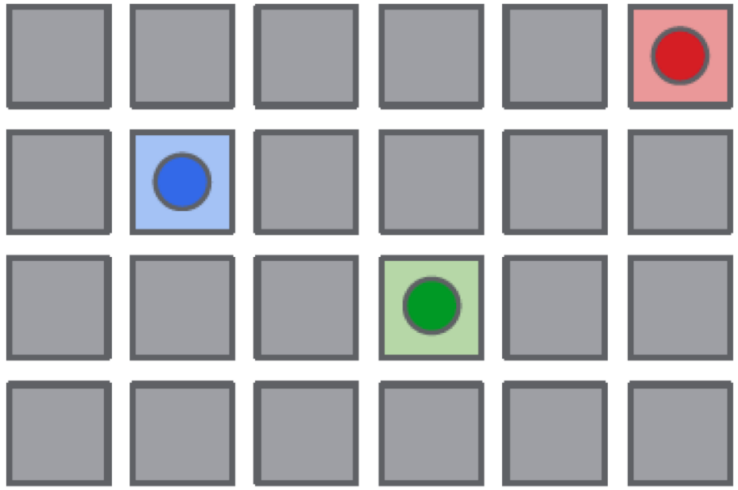
No single application/framework can manage cluster resources efficiently

General Problem

Increasing number of diverse tasks arriving



Cluster of machines with different capacity



Goal

Maximize Utilization,
while maintaining Service Level Agreement (SLA)

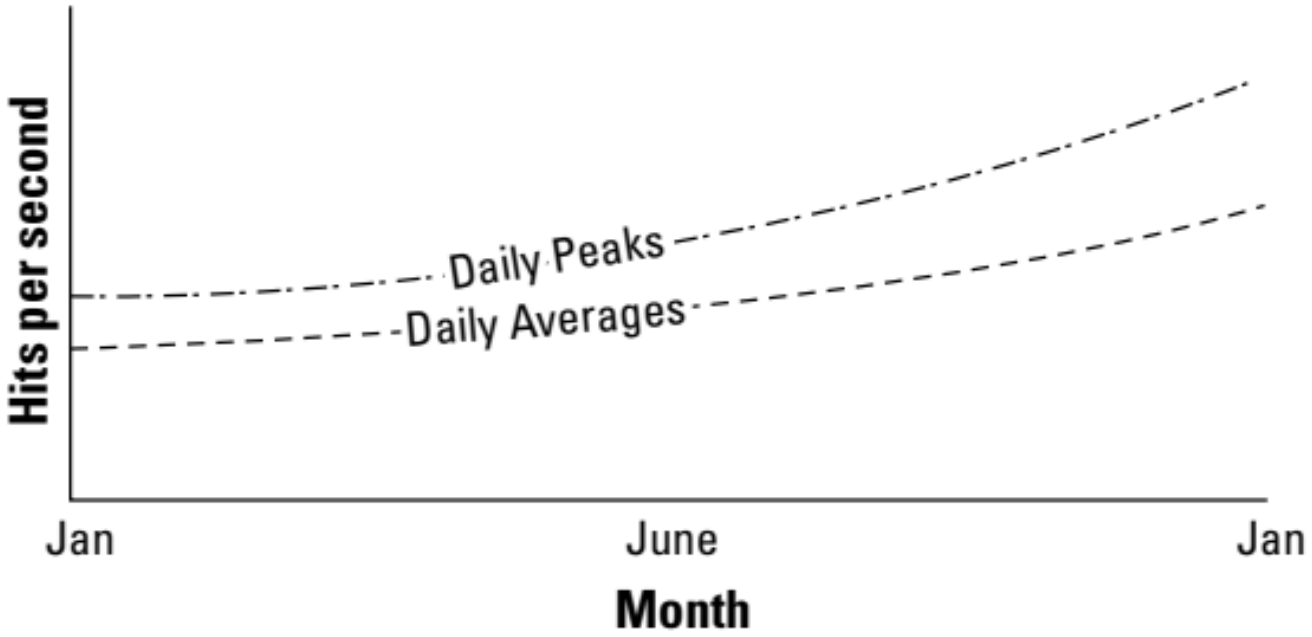
Today

- Cloud Economics
- Fairness
- Mesos
- Omega
- Borg (Google, parent of K8s)

Cloud Economics

Workload is not Consistent

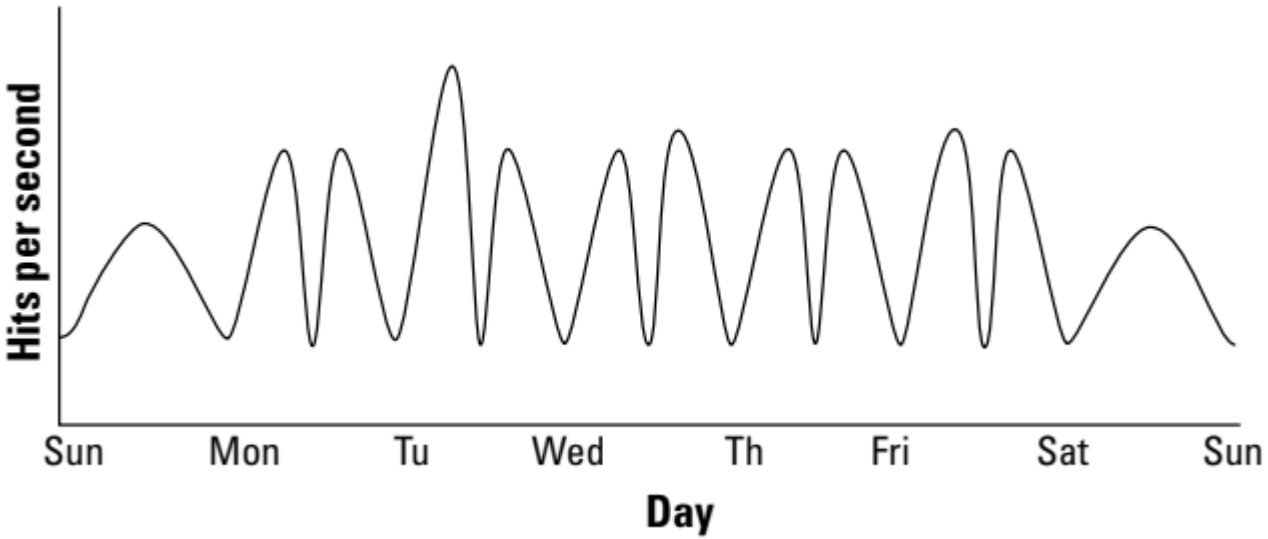
Cloud providers predict demand to plan for future expansions

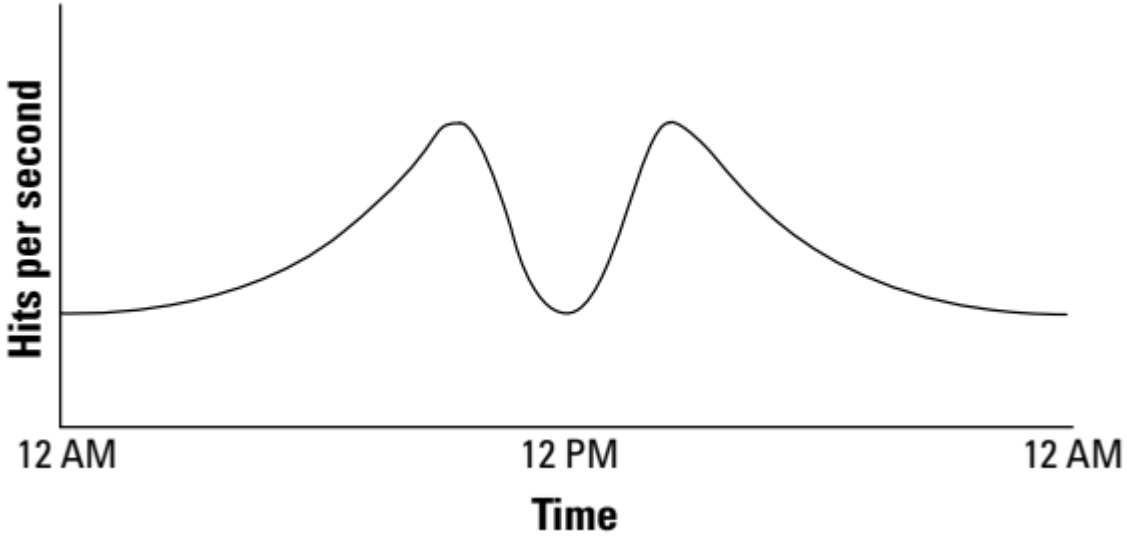


Workload is not Consistent

However peak demand could be harder to predict

- It could even vary by a minute-by-minute basis





Service Level Agreement (SLA)

Contract for service performance

- Large enterprises: Negotiate deals
- Individuals / Small Companies: Standardized SLAs

SLA Specifies

- Availability of service (uptime)
- Response time (latency)
- Reliability
- Responsibilities of each party
- Warranty

Service Level Agreement (SLA)

Availability SLA

- 99% ➔ 3.65 days/year downtime
- 99.9% ➔ 8.76 hrs/year downtime
- 99.99% ➔ 52.56 mins/year downtime

Service Level Agreement (SLA)

Availability SLA

- 99% ➔ 3.65 days/year downtime
- 99.9% ➔ 8.76 hrs/year downtime
- 99.99% ➔ 52.56 mins/year downtime

Example: Amazon S3 Availability SLA is set at 99.9%

- If uptime drops below 99.9, but above 99 ➔ Get 10% off
- If uptime drops below 99 ➔ Get 25% off

Service Level Agreement (SLA)

How to achieve 99.999% Availability ➔ Replications

- Probability of all n replicas failed $= p^n$
- Probability of at least 1 replica failed $= 1 - (1 - p)^n$

Example with p = 0.05

Number of replicas n	Probability of at least 1 server failed $P(\geq 1)$	Probability of all servers fail $P(all\ n)$	Probability of at least 1 server operational $P(\leq n-1)$
1	0.05	0.05	0.95
3	0.1426	0.000125	0.99986
5	0.2262	0.0000003125	0.9999996875
20	0.6415	9.54e-27	~1

Laws of Cloudonomics

1. Utility services costs less

Customers pay for what the use only

	Instance name	vCPUs	Memory	Network Performance	Storage	On-Demand Hourly Cost	CurrentGeneration	Potential Effective Hourly Cost (Savings %)
○	t2.nano	1	0.5 GiB	Low	EBS only	0.0058	Yes	0.0022 (63%)
○	t2.micro	1	1 GiB	Low to Moderate	EBS only	0.0116	Yes	0.0044 (62%)
○	t2.small	1	2 GiB	Low to Moderate	EBS only	0.023	Yes	0.0087 (62%)
○	t2.medium	2	4 GiB	Low to Moderate	EBS only	0.0464	Yes	0.0174 (62%)
○	t2.large	2	8 GiB	Low to Moderate	EBS only	0.0928	Yes	0.0349 (62%)
○	t2.xlarge	4	16 GiB	Moderate	EBS only	0.1856	Yes	0.0698 (62%)
○	t2.2xlarge	8	32 GiB	Moderate	EBS only	0.3712	Yes	0.1396 (62%)
○	p2.xlarge	4	61 GiB	High	EBS only	0.9	Yes	0.0000 (100%)
○	p2.8xlarge	32	488 GiB	10 Gigabit	EBS only	7.2	Yes	0.0000 (100%)
○	p2.16xlarge	64	732 GiB	20 Gigabit	EBS only	14.4	Yes	0.0000 (100%)

Laws of Cloudonomics

2. On-demand trumps forecasting

Ability to measure usage and apply pay-per-use, captures revenue and reduces cost

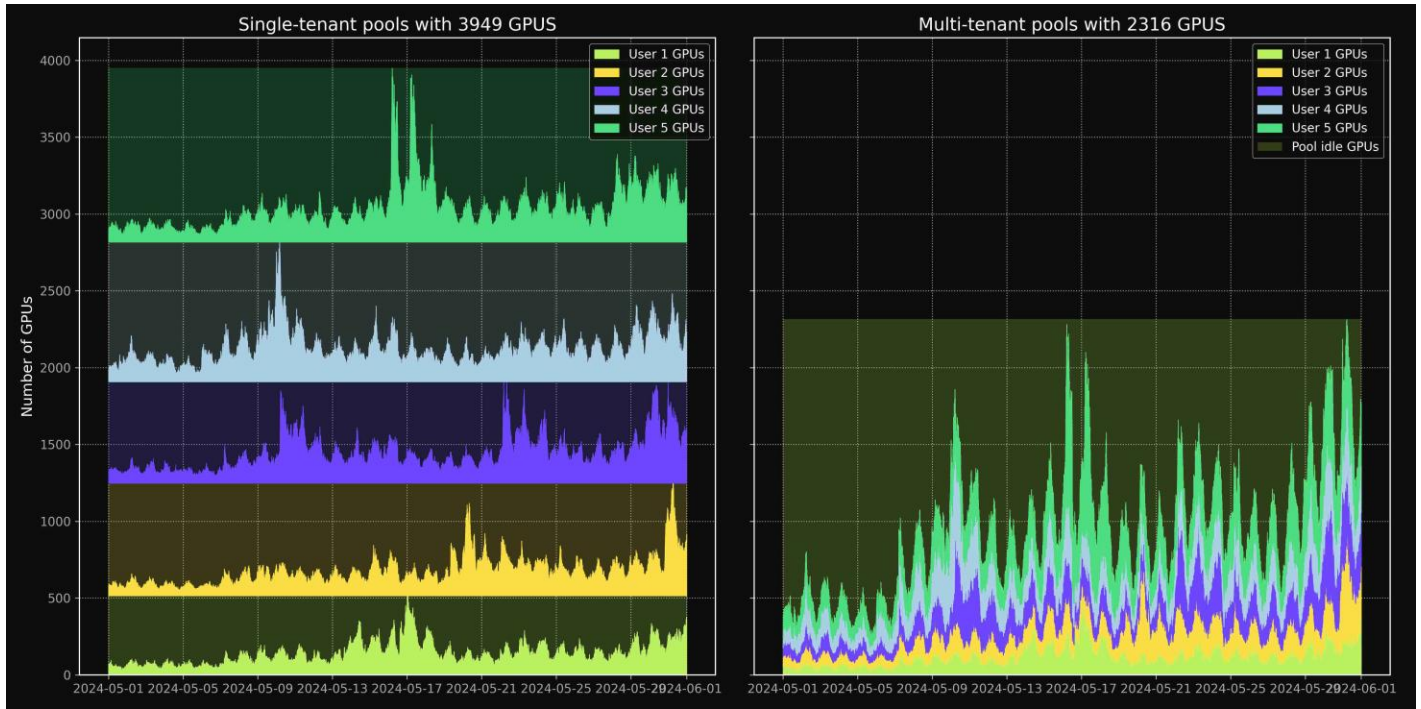
- Users track their expenses and adjust their usage

Laws of Cloudonomics

3. Peak of the sum is less than the sum of peaks

Cloud providers could deploy less capacity

- Peak usage in a multi-tenancy system is averaged over time

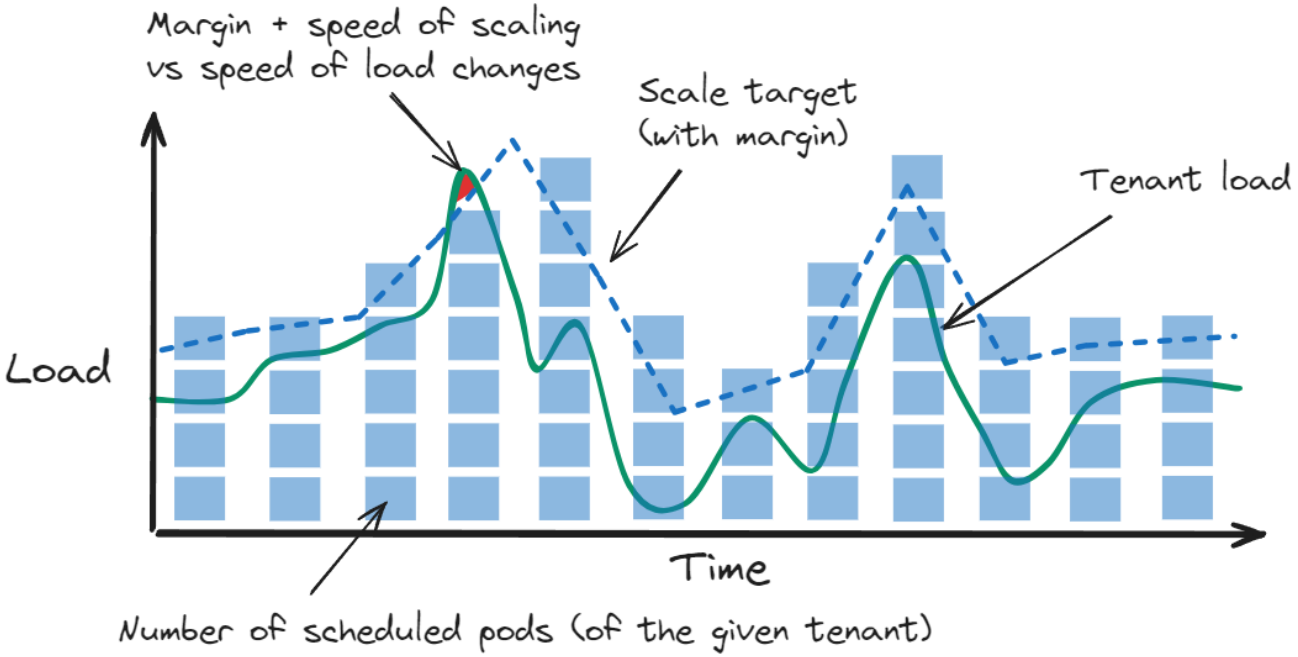


Laws of Cloudonomics

4. Aggregate demand is smoother than individual

Cloud providers can operate at higher efficiencies & lower cost

- Demand is more predictable than in a private system



Laws of Cloudonomics

5. Average unit costs are reduced by distributing fixed cost over more output units

Cloud providers can scale-up their resources with a significantly reduced price

NVIDIA H100 GPU
~40,000$

+

AWS Pricing
$39~$49 / hr

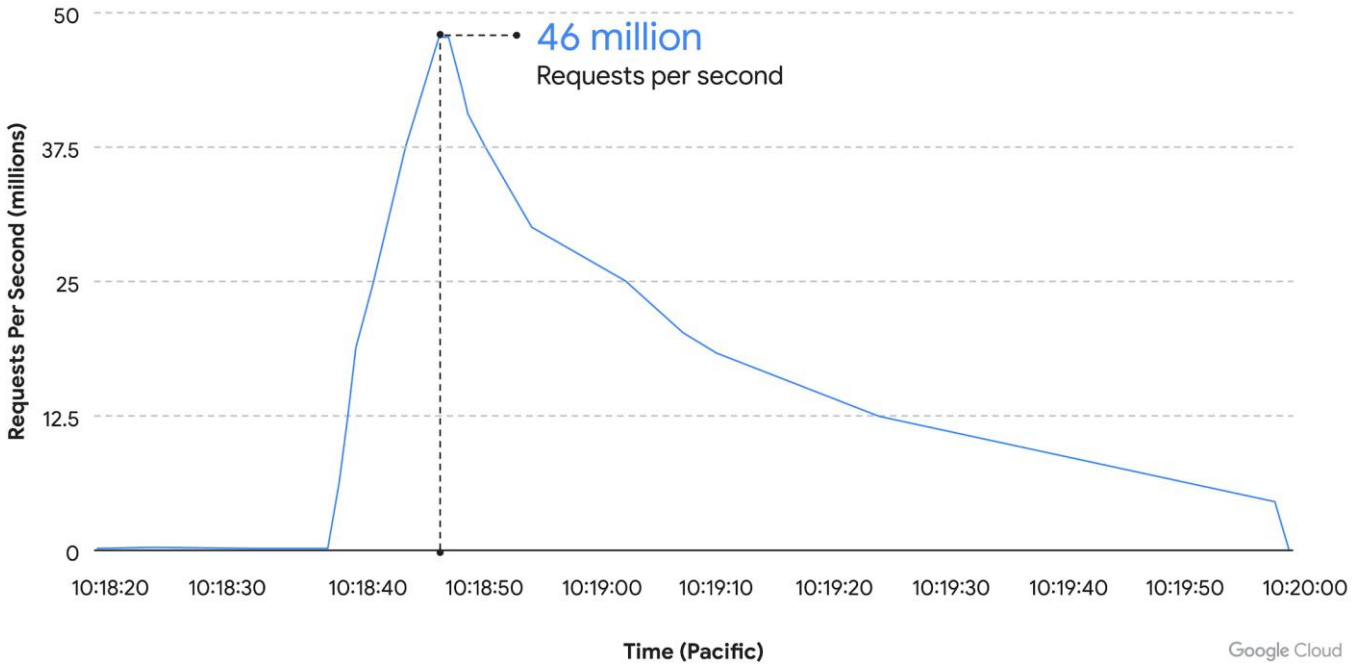
→

Return on Investment
~60 Days

Laws of Cloudonomics

6. Superiority in numbers is important against bad actors

Cloud providers can defend off bots and DDoS attacks easier compared to smaller systems



Laws of Cloudonomics

7. Parallelism is core key

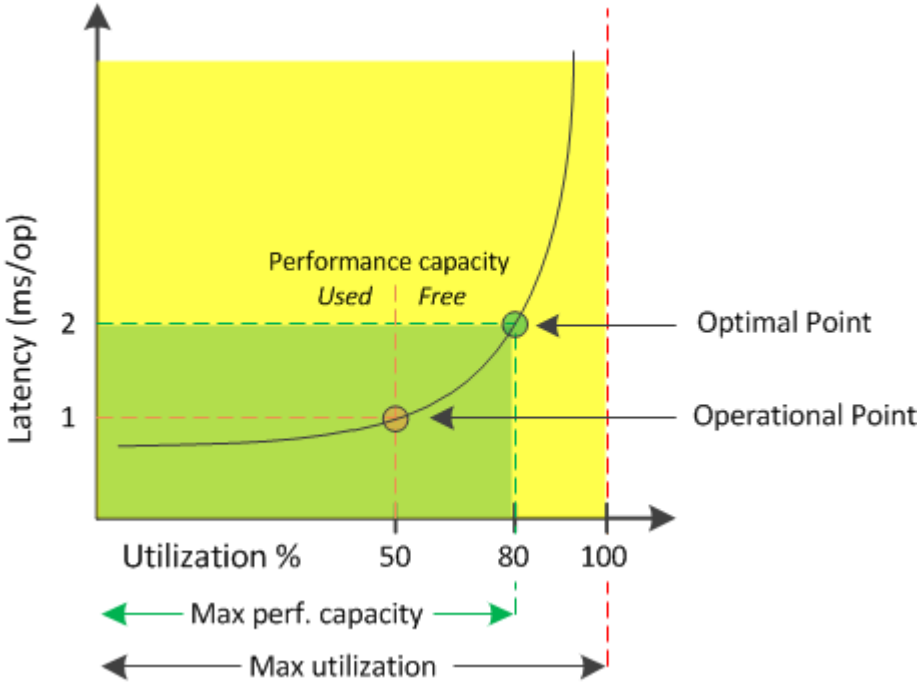
Parallel processing of a task on the cloud allows real-time businesses to respond quicker to business conditions and accelerate decision making

Laws of Cloudonomics

8. Dispersion is inverse square of latency

Low latency requires large-scale and multi-site deployments

- Reducing latency by half requires four times the number of resources

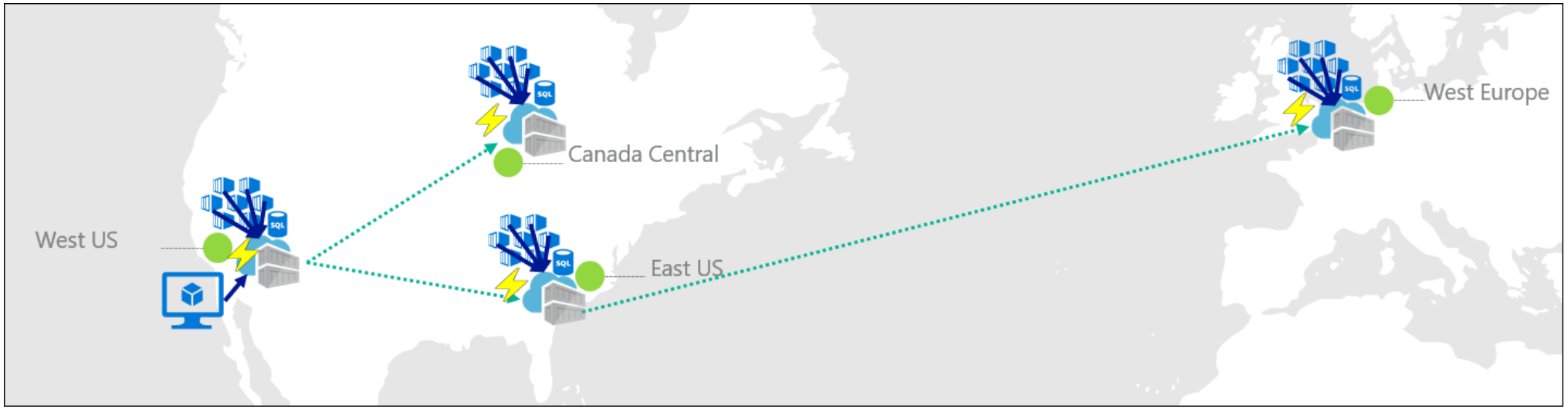


Laws of Cloudonomics

9. Avoid having all eggs in one basket

Replicated geographically dispersed datacenters are needed to achieve high reliability rates

- Hard for private systems to achieve the same level



Laws of Cloudonomics

10. Datacenters are usually located in “greenfield” sites

Cloud provides build datacenters at sites, where:

- On network backbone
- Cheap access to power & cooling
- Land is inexpensive
- Low impact on environment

Scheduling

Scheduler Role

Scheduler is responsible for resource sharing

- Efficient: Achieve high utilization
- Fair: All users have access to their paid resources
- Starvation-Free: No task is waiting indefinitely for service

Scheduler Role

Scheduler is responsible for resource sharing

- Efficient: Achieve high utilization
- Fair: All users have access to their paid resources
- Starvation-Free: No task is waiting indefinitely for service

Goal for batch systems ➔ Maximize throughput

Goal for real-time systems ➔ Meet deadlines

Goal for service systems ➔ Availability & latency

Scheduler Role

Scheduler is responsible for resource sharing

- Efficient: Achieve high utilization
- Fair: All users have access to their paid resources
- Starvation-Free: No task is waiting indefinitely for service

Goal for batch systems ➔ Maximize throughput

Goal for real-time systems ➔ Meet deadlines

Goal for service systems ➔ Availability & latency

Round Robin, FCFS, SJF

Earliest Deadline First (EDF)
Rate Monotonic Alg (RMA)

Fair Queuing

Scheduler Role

Scheduler is responsible for resource sharing

- Efficient: Achieve high utilization
- Fair: All users have access to their paid resources
- Starvation-Free: No task is waiting indefinitely for service

Goal for batch systems ➔ Maximize throughput

Goal for real-time systems ➔ Meet deadlines

Goal for service systems ➔ Availability & latency

OS Course

Round Robin, FCFS, SJF

Earliest Deadline First (EDF)
Rate Monotonic Alg (RMA)

RT version

Fair Queuing

Fair Queuing

Family of scheduling algorithms

- Achieve fairness
- Limited shared resources
- Examples
  - 1985 Round-Robin
  - 1989 Byte-weighted fair queuing
  - 1993 Weighted fair queuing

What is Fairness?

The amount of work Ω during time interval t of 2 runnable threads a and b minimizes the equation:

$$\left| \frac{\Omega_a(t)}{w_a} - \frac{\Omega_b(t)}{w_b} \right|$$

Where w_a and w_b are weights for each thread determined by importance, Quality-of-Service, SLA, and price paid

Max-Min Fairness

Max-Min Fairness

For a set of n requests each demands x_i ➔ $\{x_1, x_2, x_3, \dots, x_n\}$

- Increasing the resources allocated to request i means decreasing resources for request j, where the demand of $x_j < x_i$

Max-Min Fairness

For a set of n requests each demands x_i ➔ $\{x_1, x_2, x_3, \ldots, x_n\}$

- Increasing the resources allocated to request i means decreasing resources for request j, where the demand of $x_j < x_i$

Maximize the min share to each request not fully serviced

Max-Min Fairness

For a set of n requests each demands x_i ➔ $\{x_1, x_2, x_3, \ldots, x_n\}$

- Increasing the resources allocated to request i means decreasing resources for request j, where the demand of $x_j < x_i$

Maximize the min share to each request not fully serviced

- Small tasks ➔ gets needed resources
- Large tasks ➔ split the rest of the capacity

Max-Min Fairness

4 Tasks demands CPUs {2, 2.6, 4, 5}, and Capacity = 10

Max-Min Fairness

4 Tasks demands CPUs {2, 2.6, 4, 5}, and Capacity = 10

1. Dividing resources equally ➔ 10/4 = 2.5

Max-Min Fairness

4 Tasks demands CPUs {2, 2.6, 4, 5}, and Capacity = 10

1. Dividing resources equally ➔ 10/4 = 2.5

Task1 has excess 0.5 CPU

Unfair

Max-Min Fairness

4 Tasks demands CPUs {2, 2.6, 4, 5}, and Capacity = 10

2. Task 1 takes 2 CPUs
   Remaining capacity divided on rest of tasks ➔ 8 / 3

Task 1	Task 2	Task 3	Task 4
2	2.667	2.667	2.667

Max-Min Fairness

4 Tasks demands CPUs {2, 2.6, 4, 5}, and Capacity = 10

2. Task 1 takes 2 CPUs
   Remaining capacity divided on rest of tasks ➔ 8 / 3

Task 1	Task 2	Task 3	Task 4
2	2.667	2.667	2.667

Task 2 takes more than needed

Max-Min Fairness

4 Tasks demands CPUs {2, 2.6, 4, 5}, and Capacity = 10

3. Task 1 takes 2 CPUs
   Task 2 takes 2.6 CPUs
   Remaining capacity divided on rest of tasks ➔ 5.4 / 2

Task 1	Task 2	Task 3	Task 4
2	2.6	2.7	2.7

Start-Time Fair Queue (SFQ)

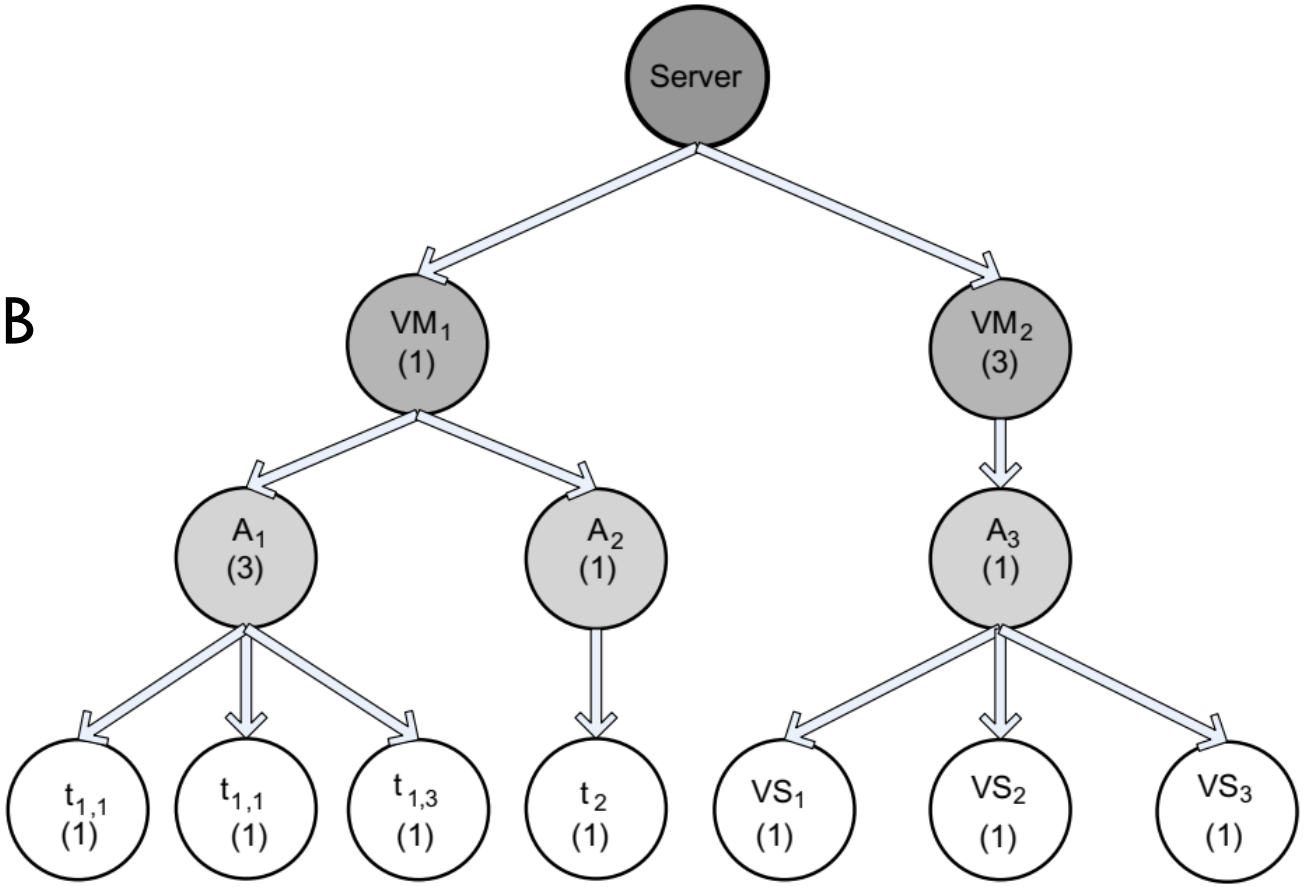
Start-Time Fair Queuing (SFQ)

Sort processes for CPU bandwidth in a tree structure

- Process/thread added to tree according to bandwidth fraction B

$$B = \frac{w_i}{\sum_{j=1}^{n} w_j}$$

- w_j is weight of child j of node i



Start-Time Fair Queuing (SFQ)

For a CPU with quantum slice q

Thread a is assigned the CPU for $q_a = q/w_a$, where w_a is thread weight

Each thread keeps track of

- Virtual time v
- Virtual Start Time at slice i ➔ $S_a(i)$
- Virtual Finish Time at slice i ➔ $F_a(i)$

Start-Time Fair Queuing (SFQ)

When scheduler is triggered for i-th step at real time τ^i

1. Calculate the virtual Start time for each thread

$$S_a(i) = \max\left[v(\tau^i), F_a(i-1)\right], where\ S_a(i) = 0$$

2. Assign CPU to thread with least virtual Start time

$$S_a(i) < S_b(i)$$

3. Calculate virtual finish time for the selected thread

$$F_a(i) = S_a(i) + q/w_a$$

4. Update virtual time

$$v(t) = \begin{cases} S_a(i), & if\ CPU\ is\ busy \\ \max_x F_x, & if\ CPU\ is\ idle \end{cases}$$

Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 1

1. Calculate Sa, Sb ➔ Sa = 0, Sb = 0

Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 1

1. Calculate Sa, Sb ➔ Sa = 0, Sb = 0
2. Select smallest S ➔ both have same value ➔ select highest weight "b"

Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 1

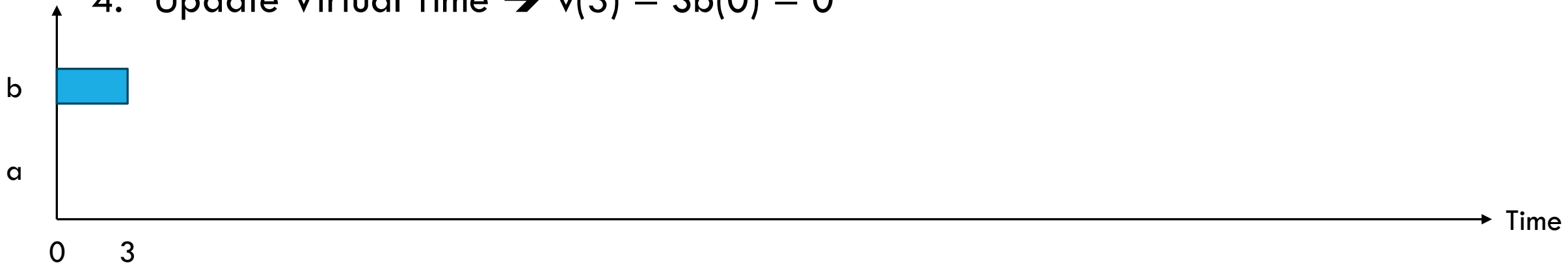
1. Calculate Start Time ➔ Sa = 0, Sb = 0
2. Select smallest S ➔ both have same value ➔ select highest weight "b"
3. Calculate Finish Time ➔ Fb = Sb + q/wb = 0 + 12/4 = 3

Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 1

1. Calculate Start Time ➔ Sa(0) = 0, Sb(0) = 0
2. Select smallest S ➔ both have same value ➔ select highest weight "b"
3. Calculate Finish Time ➔ Fb(0) = Sb(0) + q/wb = 0 + 12/4 = 3
4. Update Virtual Time ➔ v(3) = Sb(0) = 0

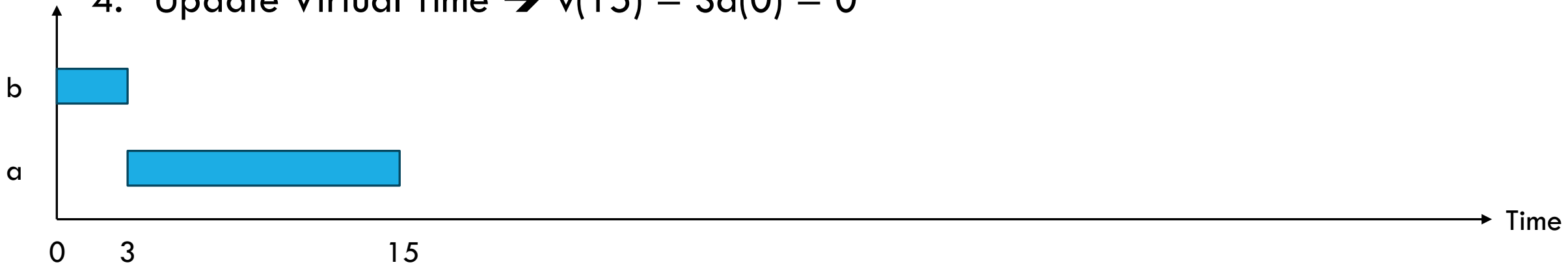


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 2

1. Calculate Start Time ➔ Sa(0) = 0, Sb(1) = max(v(3), Fb(0)) = 3
2. Select smallest S ➔ Sa(0) < Sb(1)
3. Calculate Finish Time ➔ Fa(0) = Sa(0) + q/wb = 0 + 12/1 = 12
4. Update Virtual Time ➔ v(15) = Sa(0) = 0

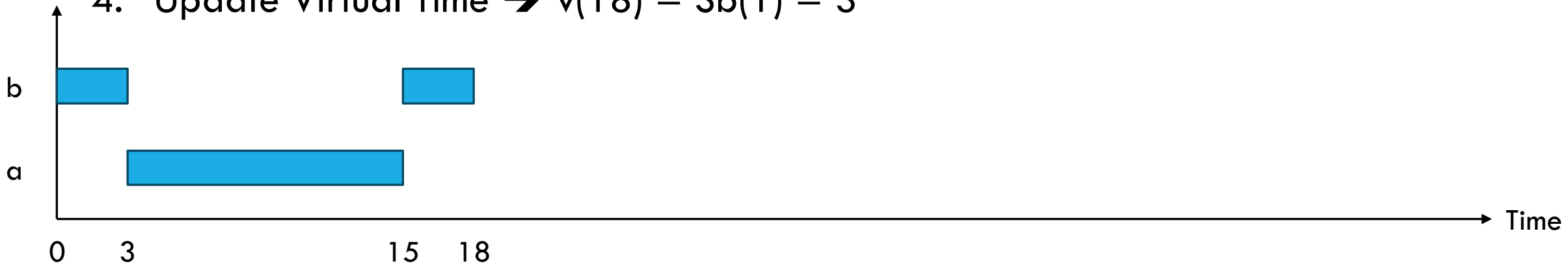


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 3

1. Calculate Start Time ➔ Sa(1) = max(v(15), Fa(0)) = 12 , Sb(1) = 3
2. Select smallest S ➔ Sb(1) < Sa(1)
3. Calculate Finish Time ➔ Fb(1) = Sb(1) + q/wb = 3 + 12/4 = 6
4. Update Virtual Time ➔ v(18) = Sb(1) = 3

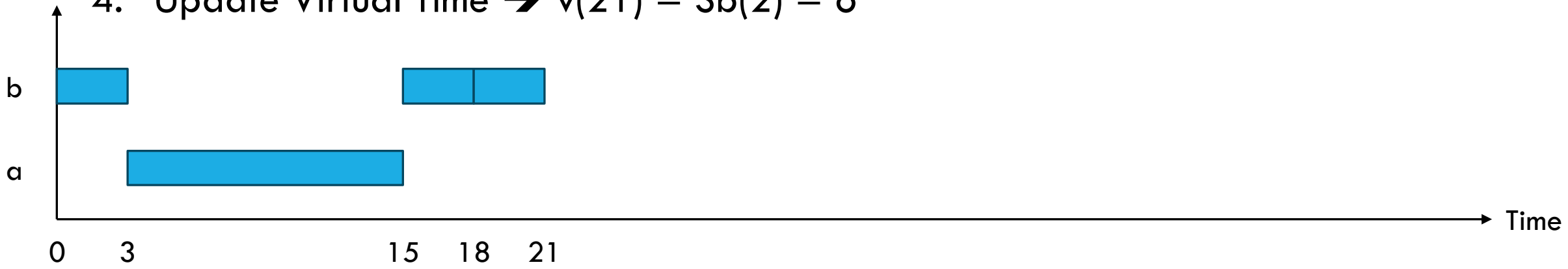


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 4

1. Calculate Start Time ➔ Sa(1) =12 , Sb(2) = max(v(18), Fb(2)) = 6
2. Select smallest S ➔ Sb(2) < Sa(1)
3. Calculate Finish Time ➔ Fb(2) = Sb(2) + q/wb = 6 + 12/4 = 9
4. Update Virtual Time ➔ v(21) = Sb(2) = 6

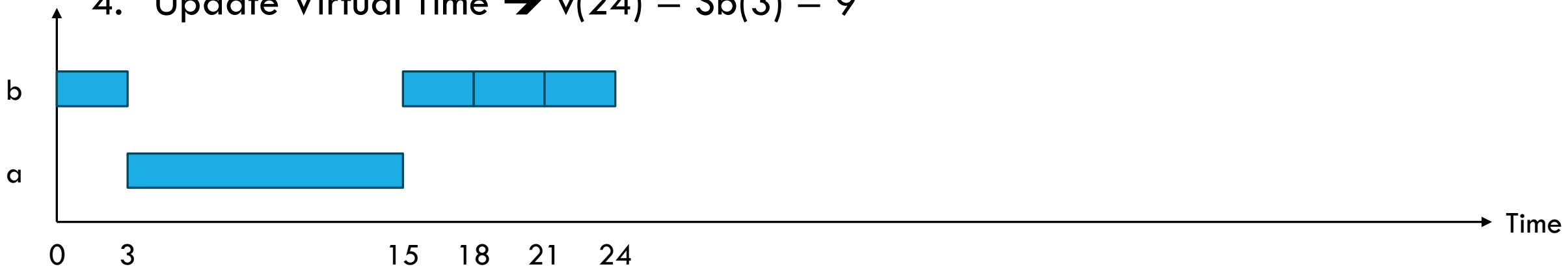


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 5

1. Calculate Start Time ➔ Sa(1) =12 , Sb(3) = max(v(21), Fb(2)) = 9
2. Select smallest S ➔ Sb(3) < Sa(1)
3. Calculate Finish Time ➔ Fb(3) = Sb(3) + q/wb = 9 + 12/4 = 12
4. Update Virtual Time ➔ v(24) = Sb(3) = 9

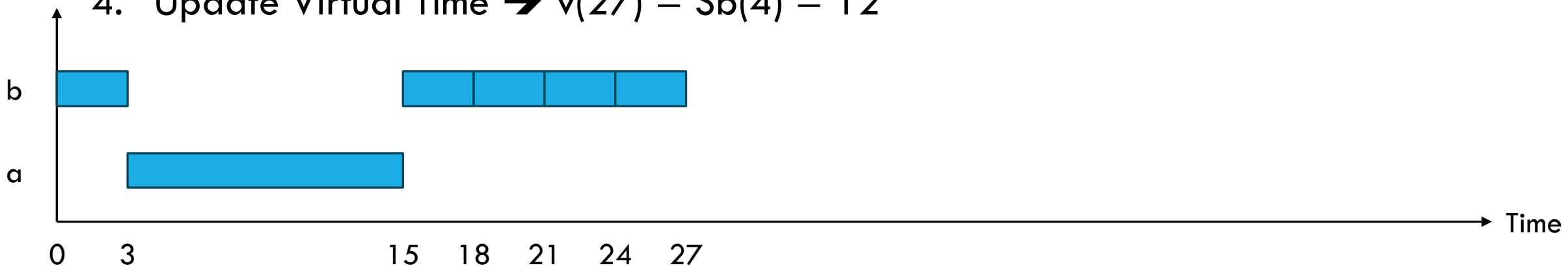


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 6

1. Calculate Start Time ➔ Sa(1) =12 , Sb(4) = max(v(24), Fb(3)) = 12
2. Select smallest S ➔ both have same value ➔ select highest weight "b"
3. Calculate Finish Time ➔ Fb(4) = Sb(4) + q/wb = 12 + 12/4 = 15
4. Update Virtual Time ➔ v(27) = Sb(4) = 12

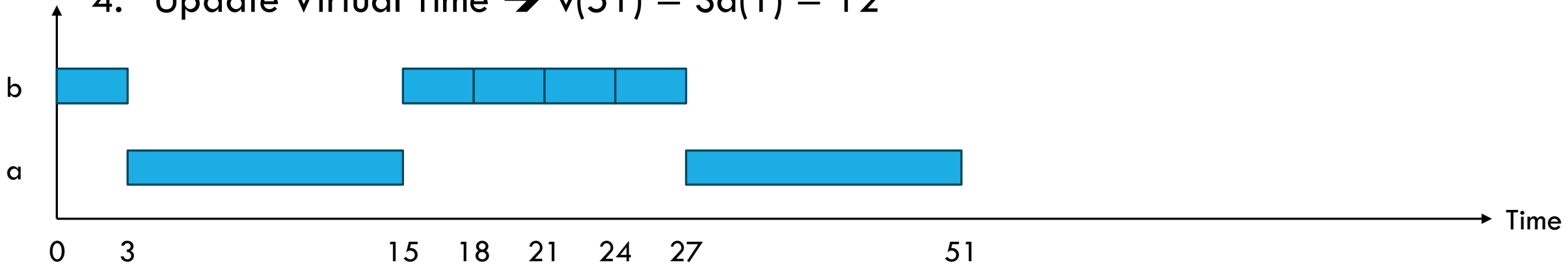


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 7

1. Calculate Start Time ➔ Sa(1) =12 , Sb(5) = max(v(27), Fb(4)) = 15
2. Select smallest S ➔ Sa(1) < Sb(5)
3. Calculate Finish Time ➔ Fa(1) = Sa(1) + q/wa = 12 + 12/1 = 24
4. Update Virtual Time ➔ v(51) = Sa(1) = 12

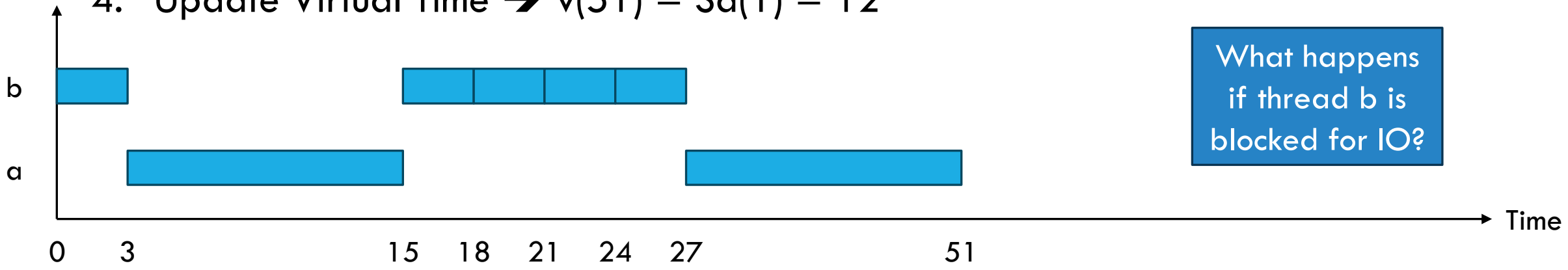


Start-Time Fair Queuing (SFQ)

A CPU with $q = 12$, and 2 threads {a, b} with $w_a = 1, w_b = 4$

Iteration 7

1. Calculate Start Time ➔ Sa(1) =12 , Sb(5) = max(v(27), Fb(4)) = 15
2. Select smallest S ➔ Sa(1) < Sb(5)
3. Calculate Finish Time ➔ Fa(1) = Sa(1) + q/wa = 12 + 12/1 = 24
4. Update Virtual Time ➔ v(51) = Sa(1) = 12



What happens if thread b is blocked for IO?

Mesos

Mesos

Frameworks such as MapReduce, HDFS, Spark run diverse set of applications

- It is difficult for a single framework to manage all resources efficiently

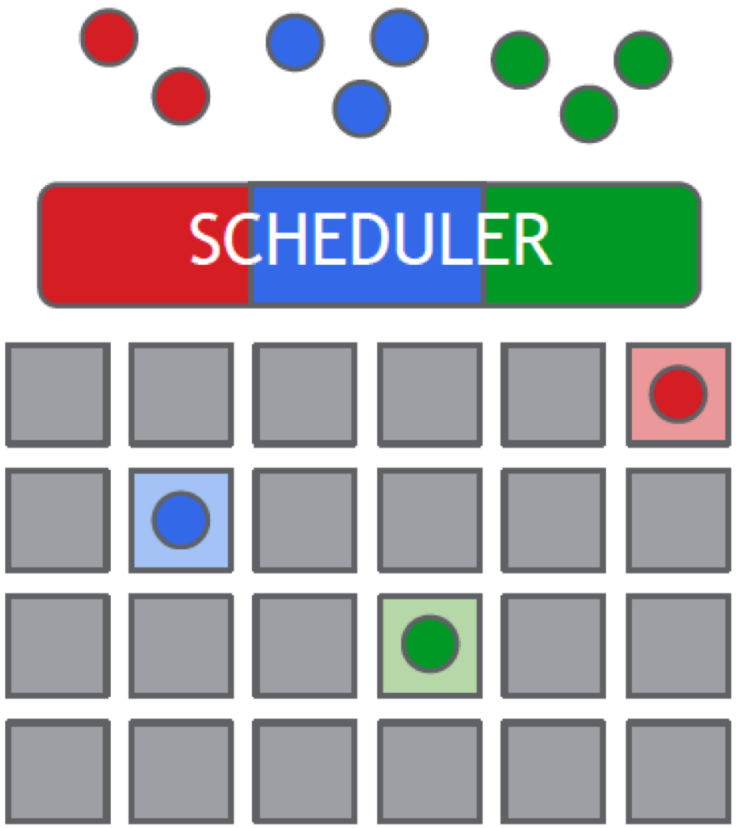
Mesos

Centralized "Monolithic" scheduler

- Assign resources using a single scheduler

Cons

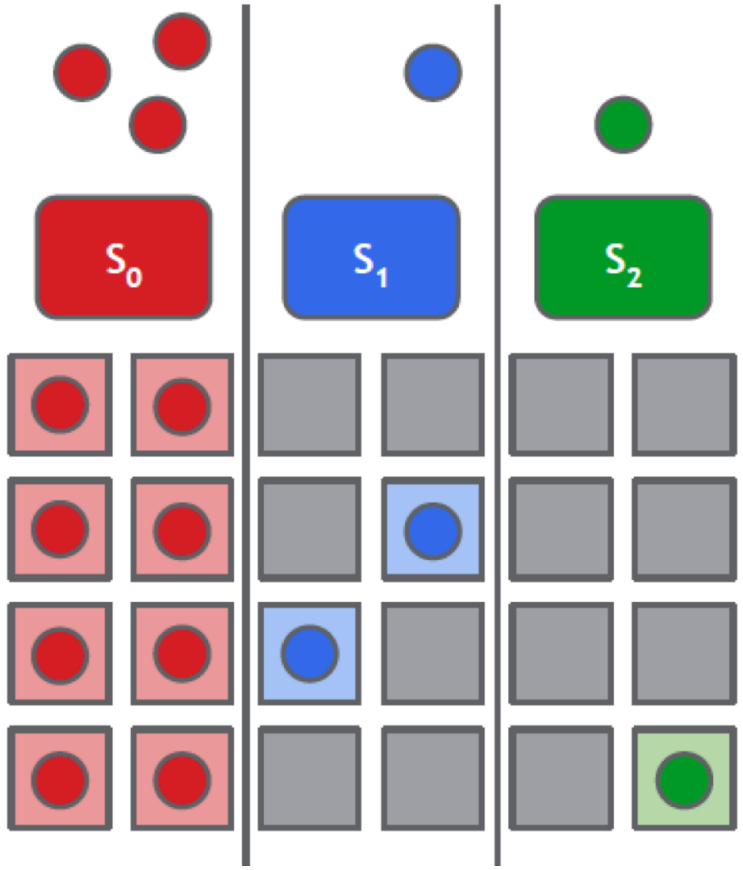
- Complexity
- Hard to diversify
- Scalability bottleneck



Mesos

Statistical Partition

- Assign set of resources to framework based on statistical need
  - X machines for MapReduce
  - MapReduce schedules MapReduce tasks



Mesos

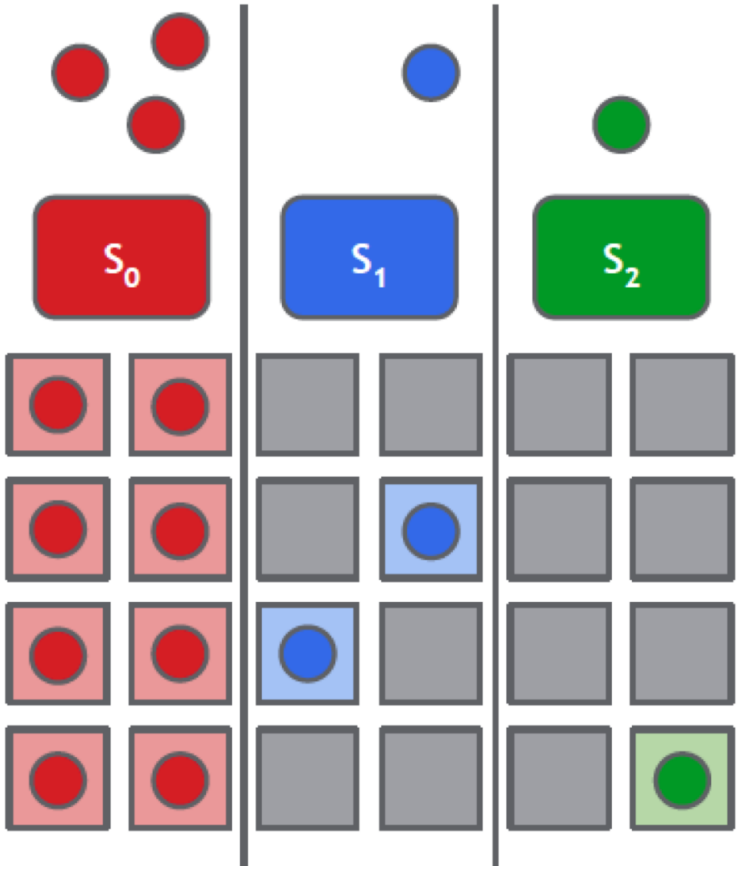
Statistical Partition

- Assign set of resources to framework based on statistical need
  - X machines for MapReduce
  - MapReduce schedules MapReduce tasks

Cons

- Low utilization
- Demand changes in real-time

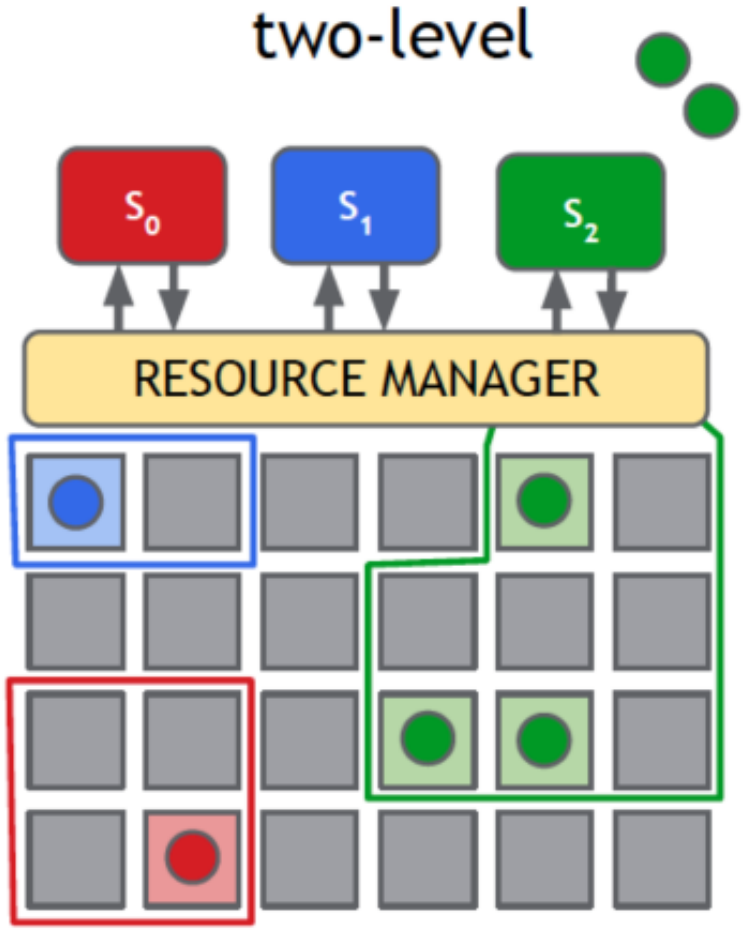
Solution ➔ Mesos



Mesos

Resource Manager Layer

- Fine-grained sharing
- Frameworks access resources through Mesos API



Mesos

2 Core Concept

1. Fine-Grained Sharing

- Assign resources on the task level
  - Instead of dedicating servers for MapReduce
  - Assign needed resources for each MapReduce task separately
- Improve utilization, latency, and locality

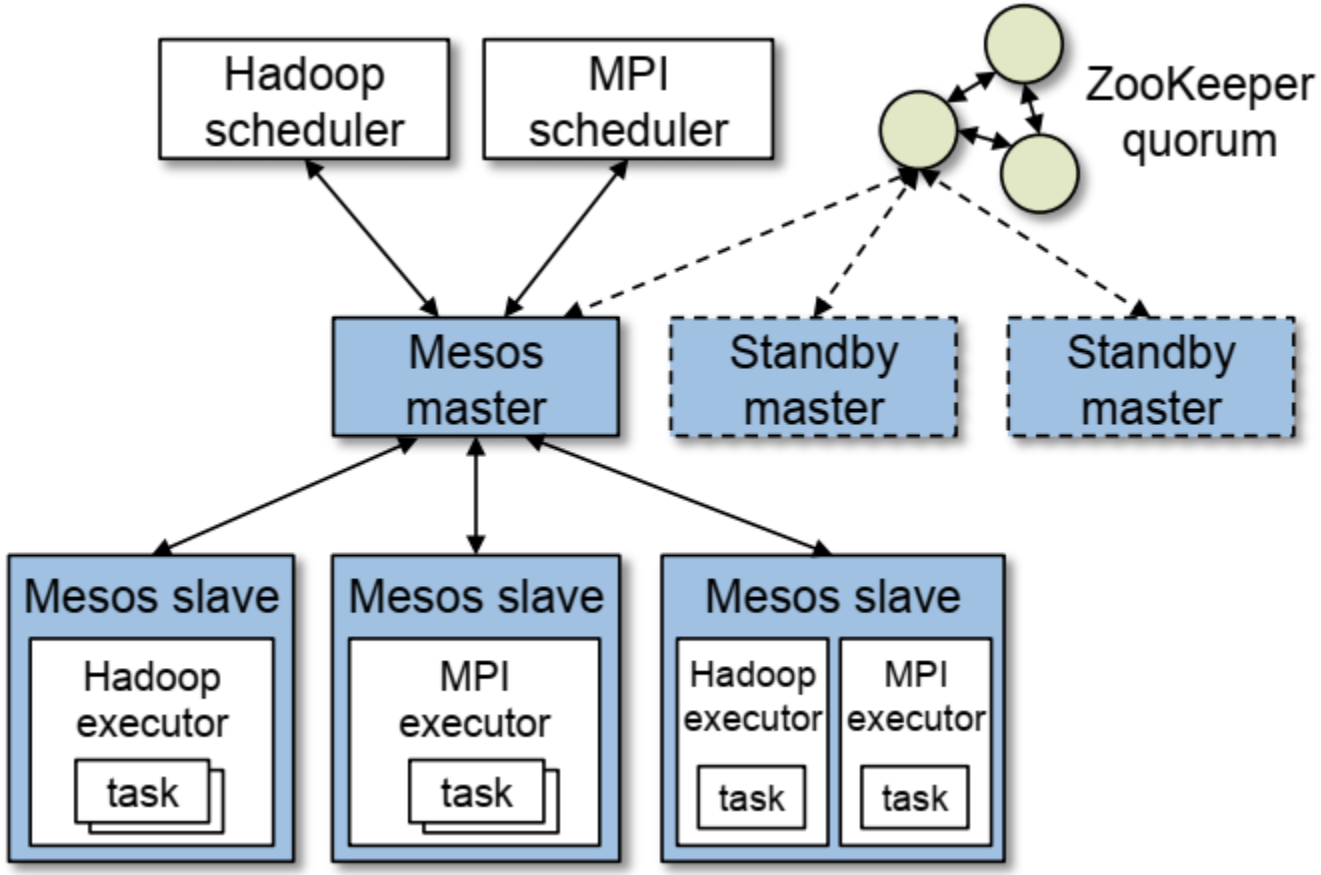
Mesos

2 Core Concept

1. Fine-Grained Sharing
2. Resource Offers
   - Decide available resources for a framework
     - Assign resources to frameworks one at a time to avoid racing condition
       ➔ Pessimistic Concurrency Control
   - Let frameworks decide which available resource to use

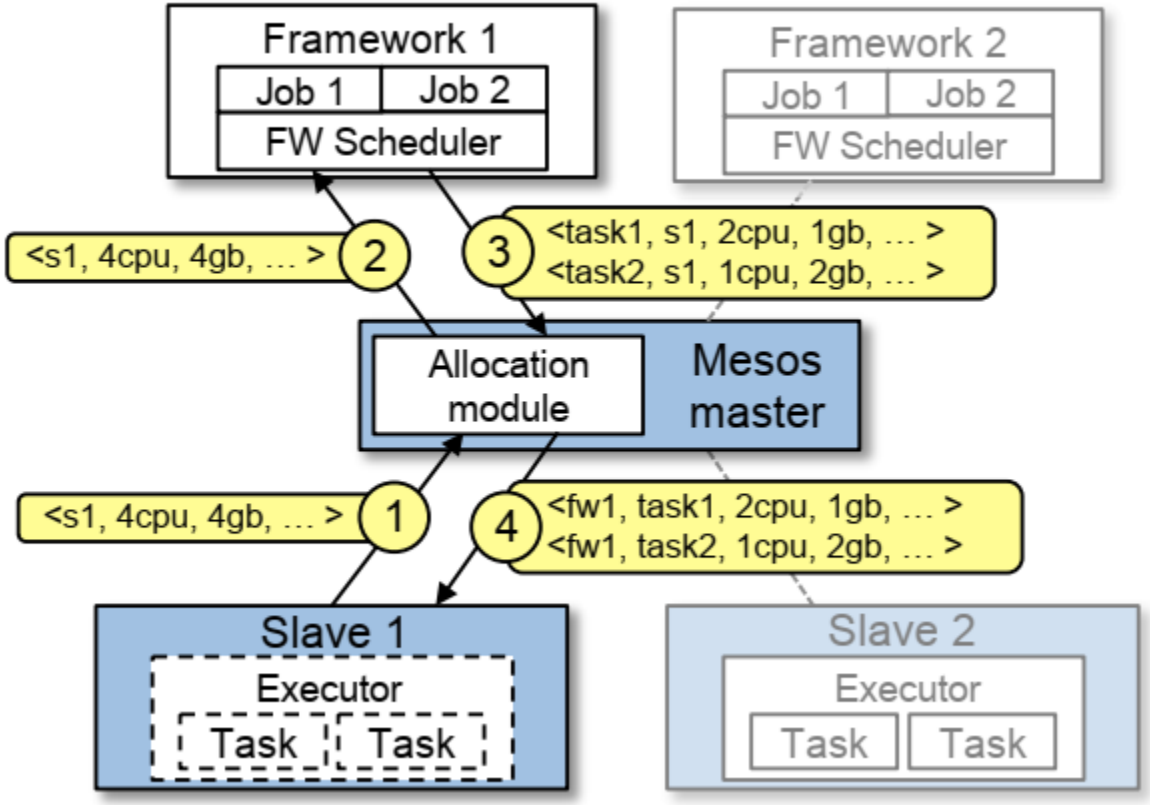
Mesos Architecture

1. Mesos Master is a replicated controller
2. Each task runs in a Mesos slave instance
3. Framework scheduler interacts with Mesos master to create/run tasks



Mesos Architecture

1. When a task requires resources, it sends request to Mesos Master
2. Mesos Master sends request & available resources to framework scheduler
3. Framework scheduler decides which resources to use
4. Mesos Master assign resources to task



Omega

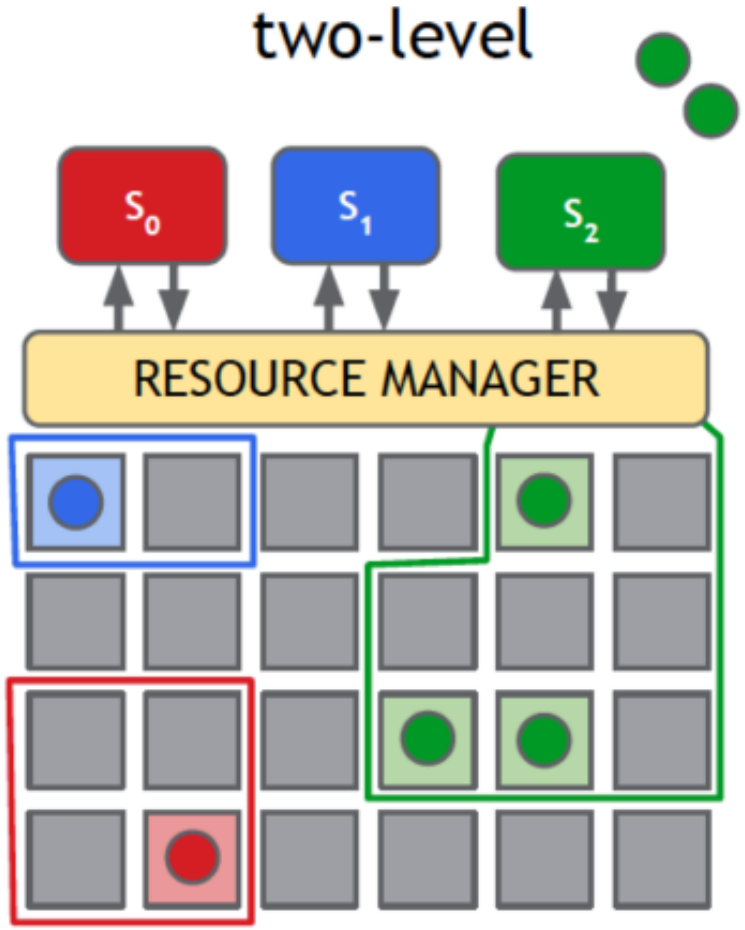
Omega

Mesos ➔ Two-Level Scheduler

Cons

- Pessimistic Concurrency Control
  - If a framework scheduler is slow, it slows the whole system
- Information Hiding
  - Framework does not have access to all the cluster state

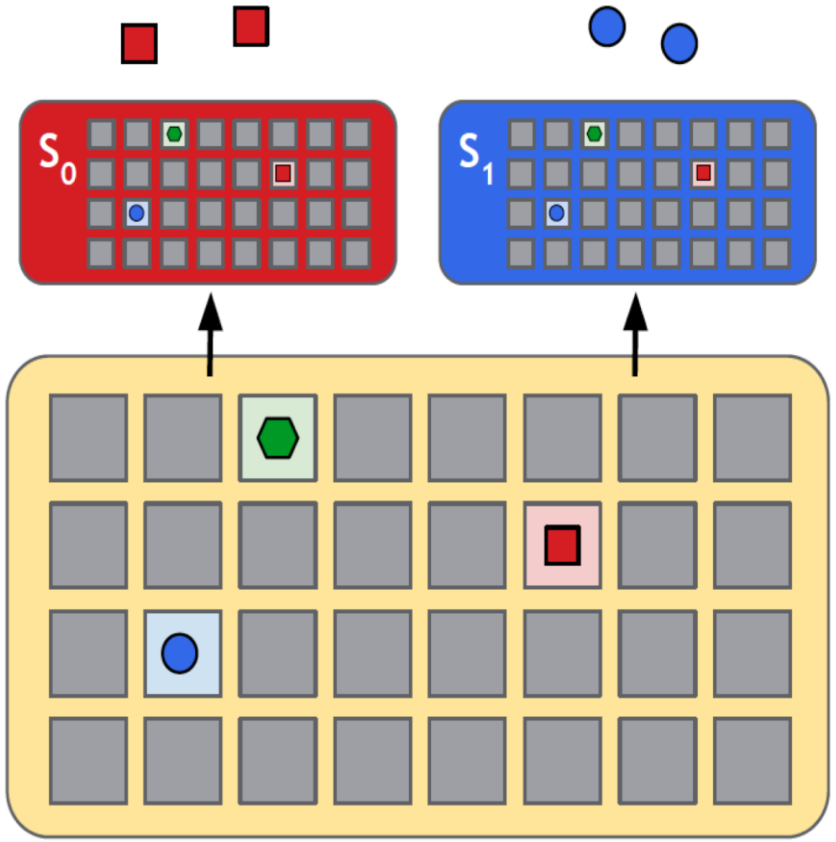
Solution ➔ Omega



Omega

Lock-free optimistic concurrency control

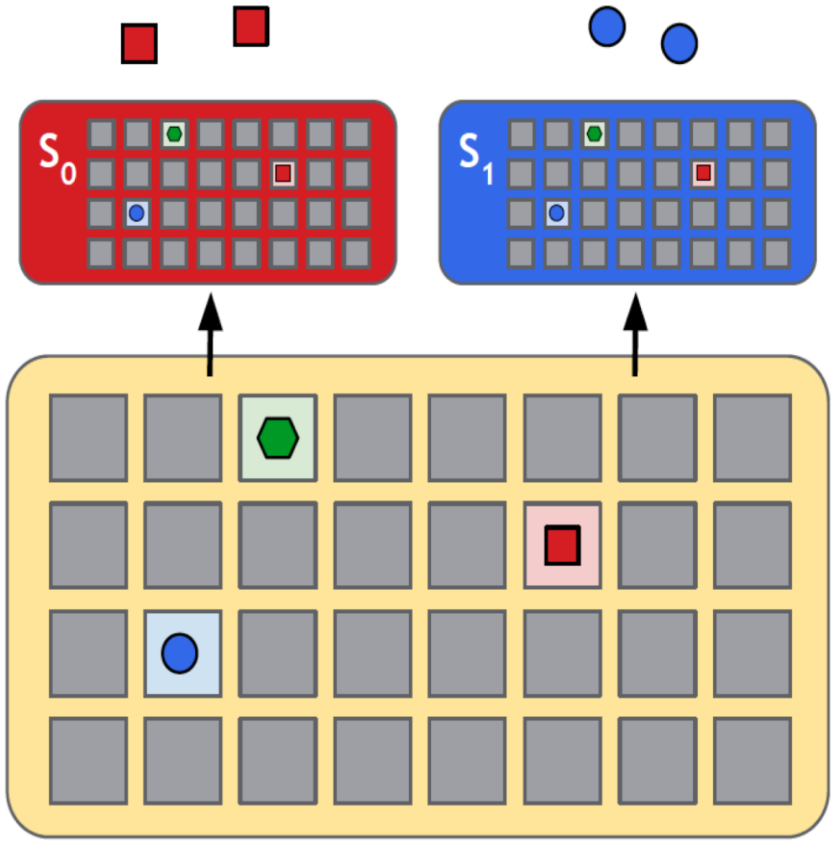
- No centralized scheduler
- Each framework has its own scheduler
- Each scheduler has a local copy of the resource allocations “Cell State”



Omega

Shared-State Scheduling

1. Framework scheduler updates its local copy of cell state
2. Framework scheduler makes a decision to assign resource to task
3. Updates the shared cell state in an atomic commit
4. Omega confirms if commit succeeds
   - At most one commit will succeed



Omega

Works well for

1. 1,000s tasks scheduled on 10,000s machines
   ➔ fewer scheduling conflicts

Omega

Works well for

1. 1,000s tasks scheduled on 10,000s machines
   ➔ fewer scheduling conflicts
2. Most tasks are “batch” tasks ➔ Successes when finishes
   lower number of “service” tasks ➔ Runs forever but consumes too many resources

Omega

What is still missing

1. How to consider task priority? ➔ Framework schedulers agree on how handle that

Omega

What is still missing

1. How to consider task priority? ➔ Framework schedulers agree on how handle that
2. What happens if task fails?
3. What happens if machine fails?
4. What happens if a machine failed 10 times in the last hour?

Omega

What is still missing

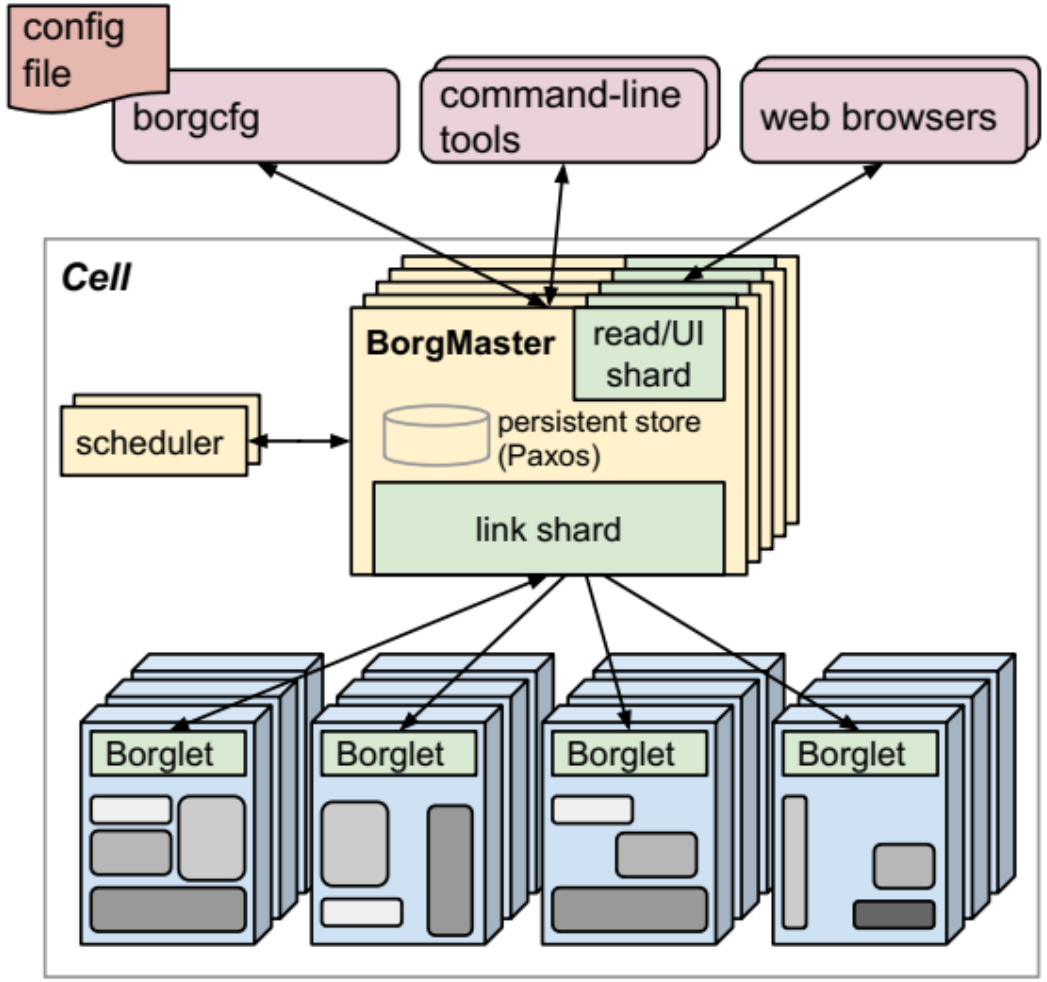
1. How to consider task priority? ➔ Framework schedulers agree on how handle that
2. What happens if task fails?
3. What happens if machine fails?
4. What happens if a machine failed 10 times in the last hour?
5. Can we still assign tasks all machines have other tasks?

Borg

Borg

Goal

- High Utilization
- Efficient Task-Packing
- Over-commitment
- Performance Isolation
- Min fault-recovery time
- Reduce correlated failures



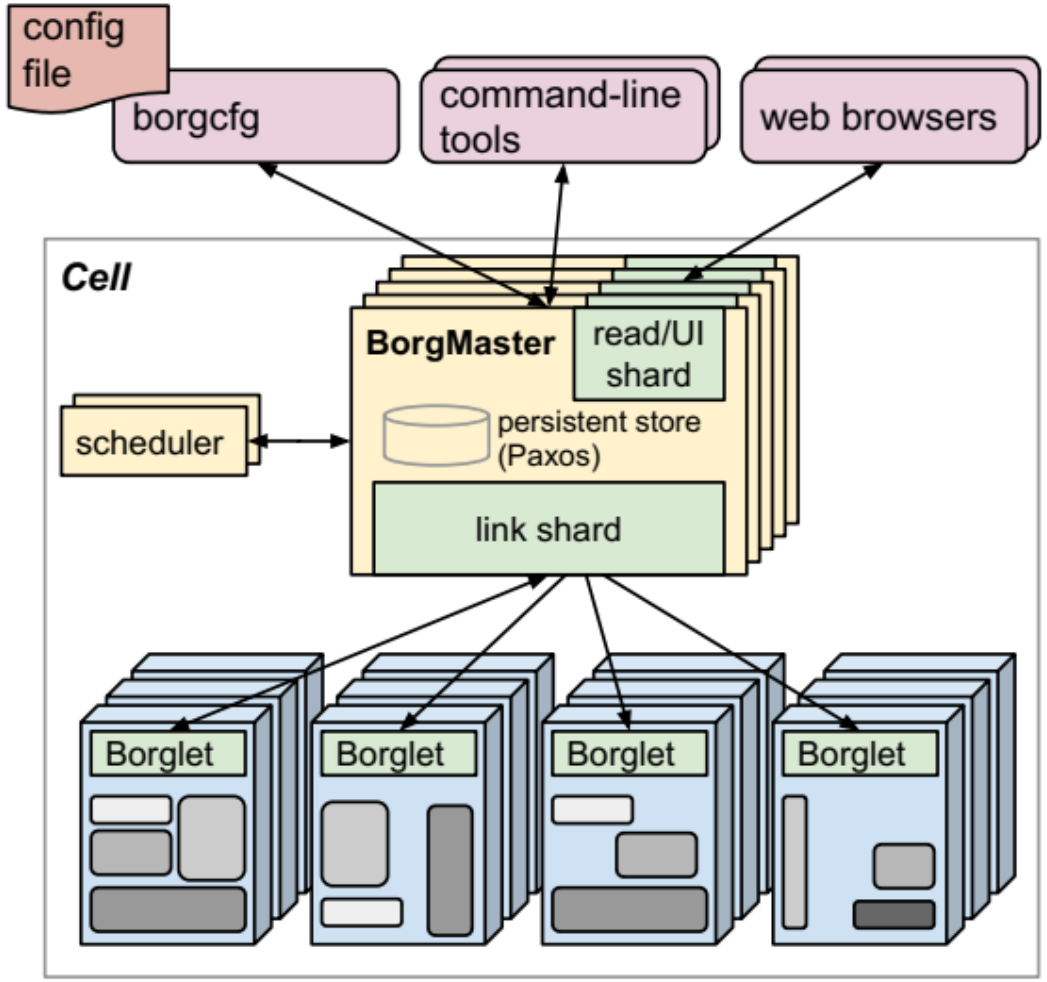
Borg

Cluster

- 10,000s Machines
- Different capacities

Cell

- Set of machines
- Managed as a single unit



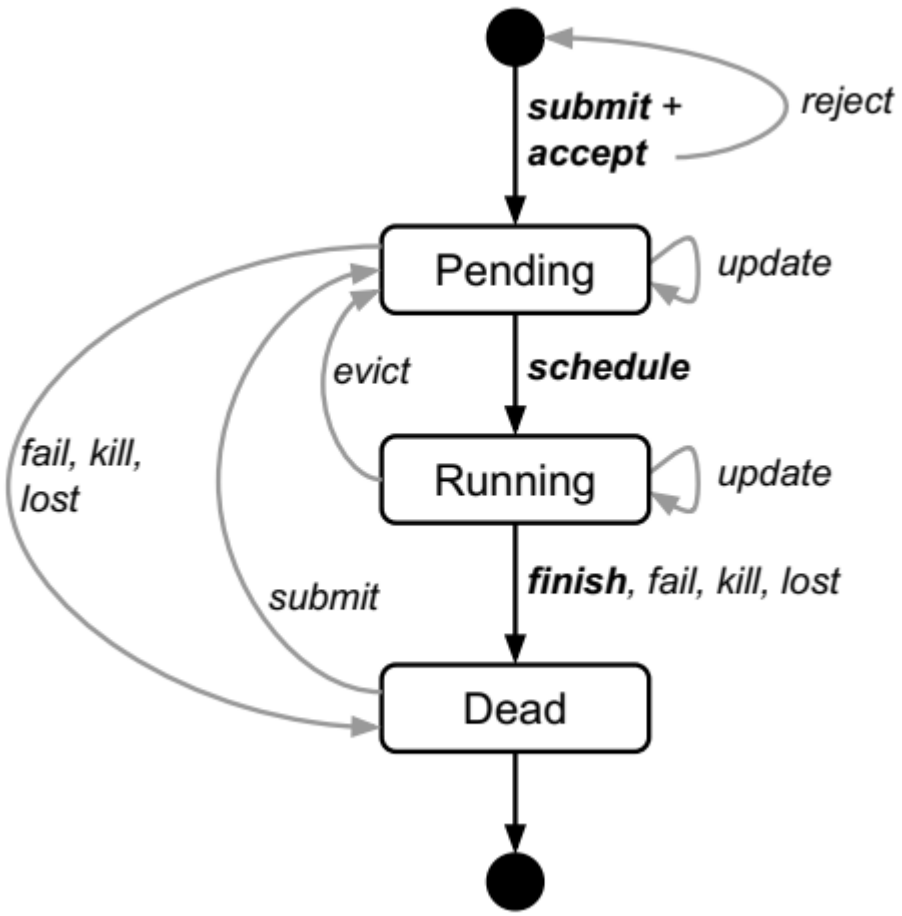
Borg Workload

Service “prod” tasks

- Gmail
- Google Docs
- Search Engine

Batch “non-prod” tasks

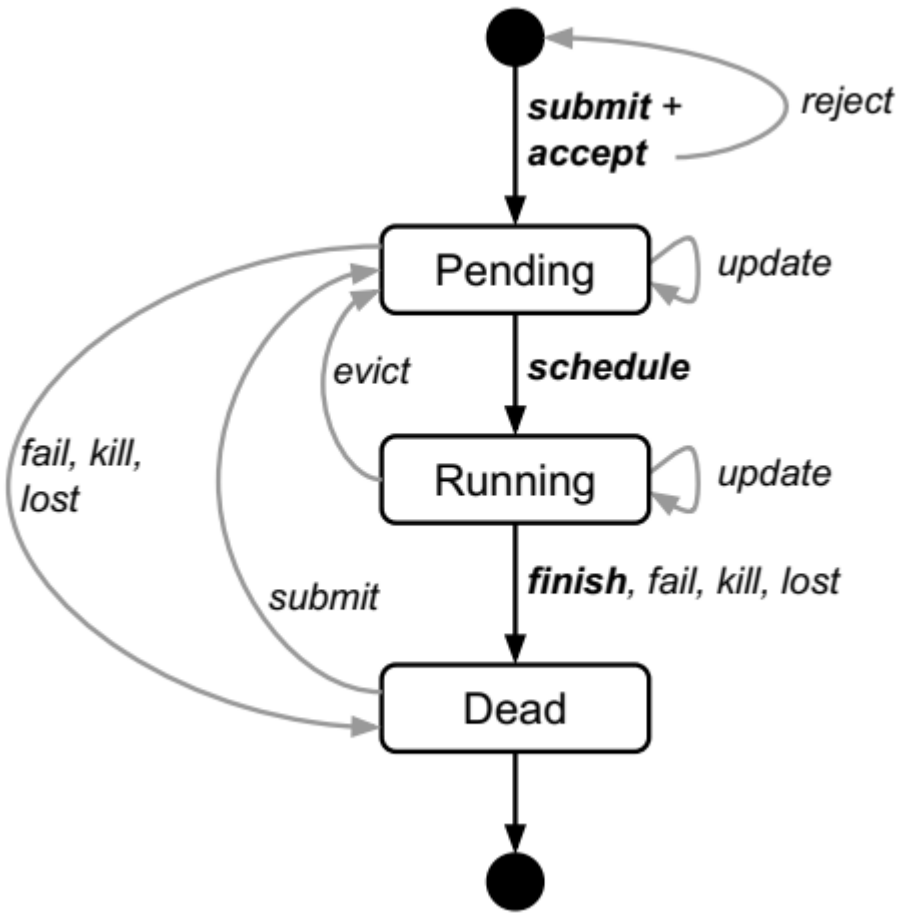
- MapReduce
- FlumeJava
- Pregel



Borg Workload

Job

- Task with required resources
- Binary & data files
- Runs on a single cell
- Comm with Borg through gRPC
- Sends health state periodically to Borg

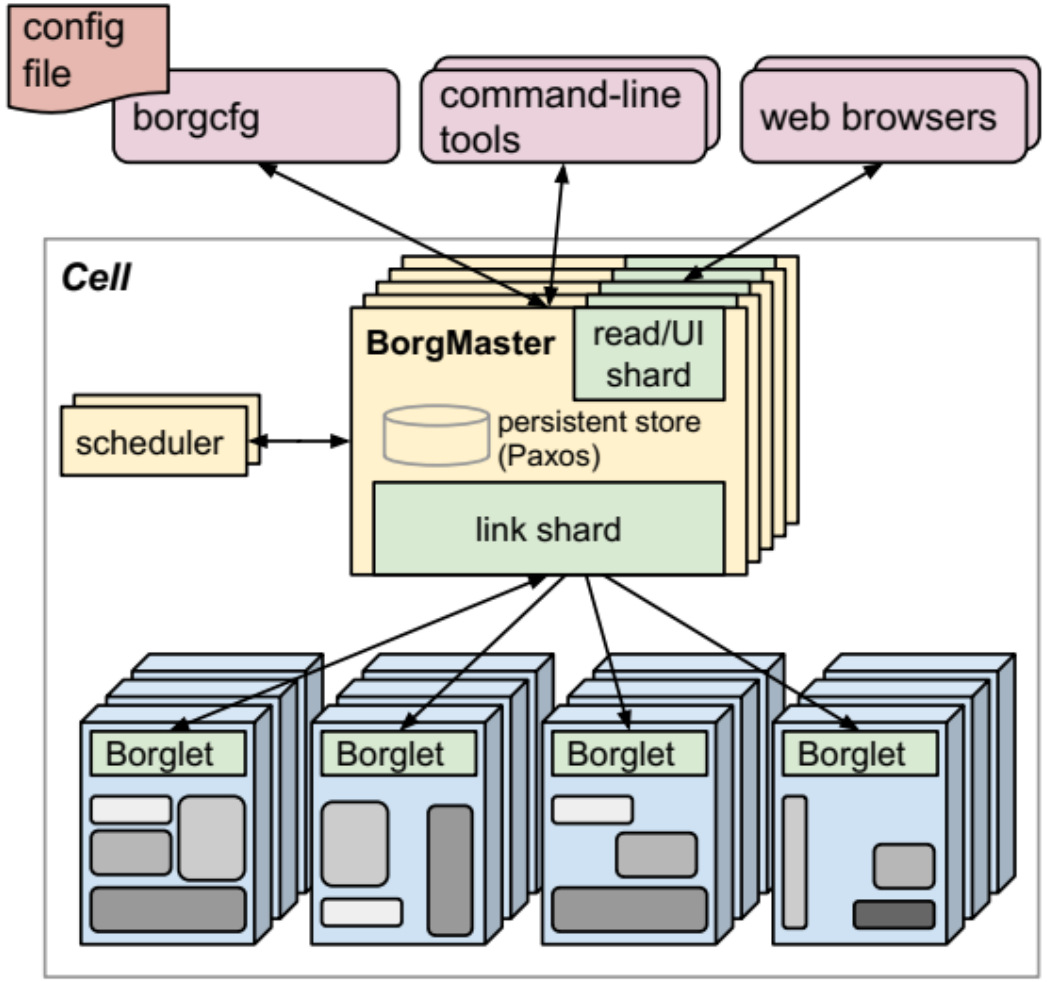


Borg Architecture

BorgMaster

- Centralized controller
- 2 separate processes
- Schedules jobs for a single cell
  - Create Jobs
  - Manage Jobs
  - Health checks & performance metrics

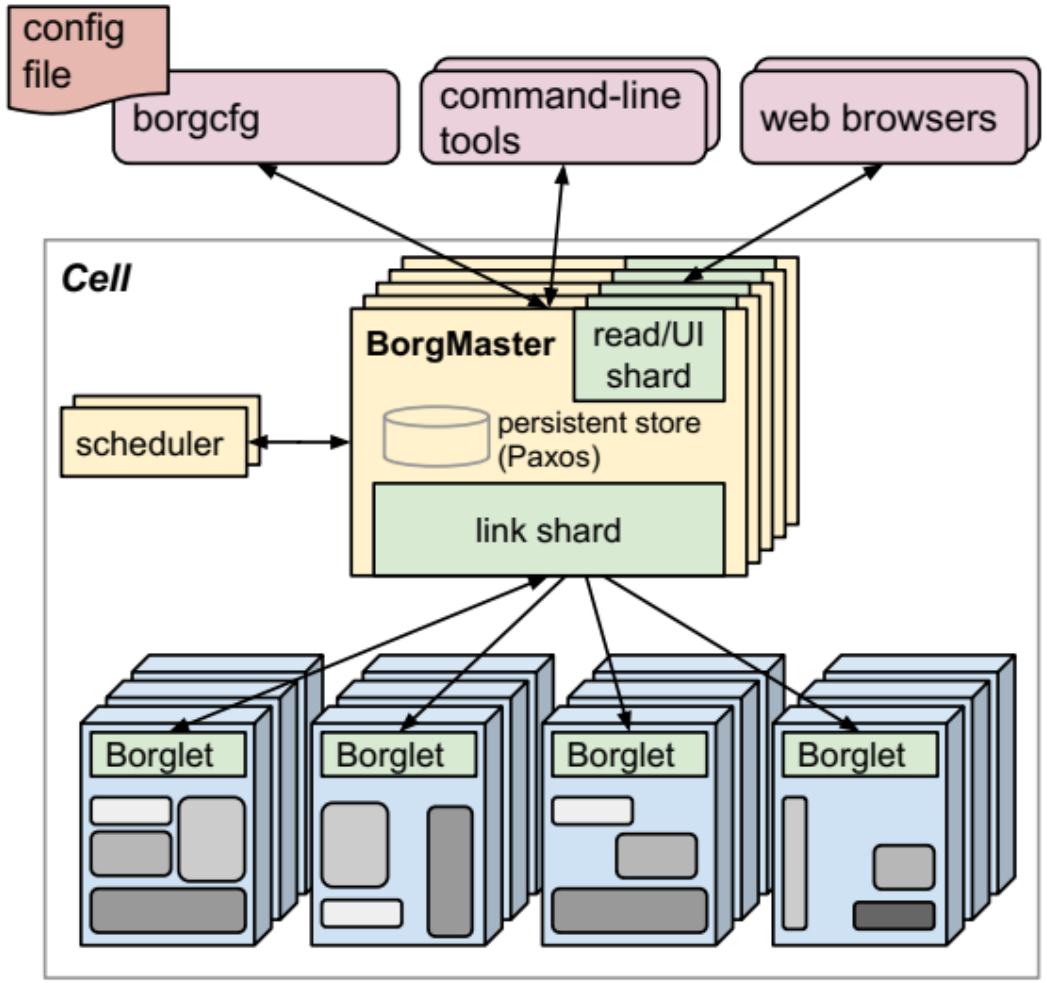
- Replicated using Paxos for fault-tolerance



Borg Architecture

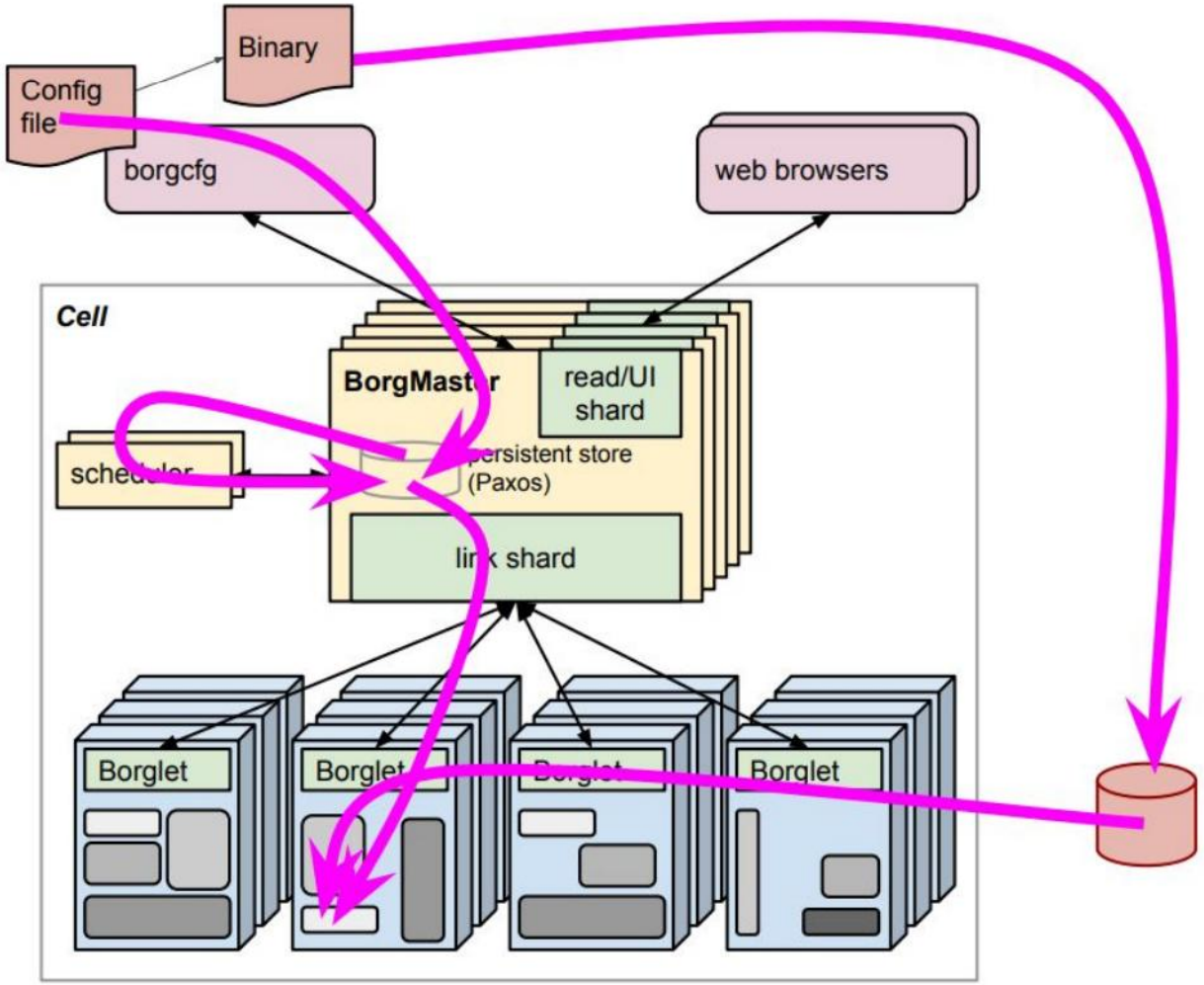
Borglet

- Linux Process
- Each machine runs a Borglet
- Communicate jobs to run on the machine
  - Start/stop job
  - Manage local resources
  - Report health to BorgMaster



Borg Architecture

1. User compiles binary version of task and config file
2. User submit job to BorgMaster using CLI/Web
3. BorgMaster add job to pending queue (Paxos store)
4. Scheduler periodically scans queue to schedule jobs on Borglets
   1. Assigns rank (priority)
   2. Round Robin jobs of same rank
   3. Feasibility check & scoring
5. Borglet pull binary & start job



Kubernetes

Open-source cluster manager ➔ Derived from Borg

Runs on Google Compute Cloud

Borg	Kubernetes
BorgMaster	Kube-api-server / etcd
Scheduler	Kube-scheduler
Borglet	Kubelet
Alloc	Pod
Borg Container	Docker
Cell	Namespace
Job / Task	Pod / Deployment / Labels

Resources

- Reference Book: Chapter 6
- 2011, Mesos: A Platform for Fine-Grained Resource Sharing in Data Center, UC Berkeley
  - https://www.youtube.com/watch?v=z_d7Wz_9QLI
- 2013, Omega: flexible, scalable schedulers for large compute clusters Google
  - https://www.youtube.com/watch?v=XsXlm4wmB6o
- 2015, Large-scale cluster management at Google with Borg, Google
  - https://www.youtube.com/watch?v=7MwxA4Fj2I4

Thank you